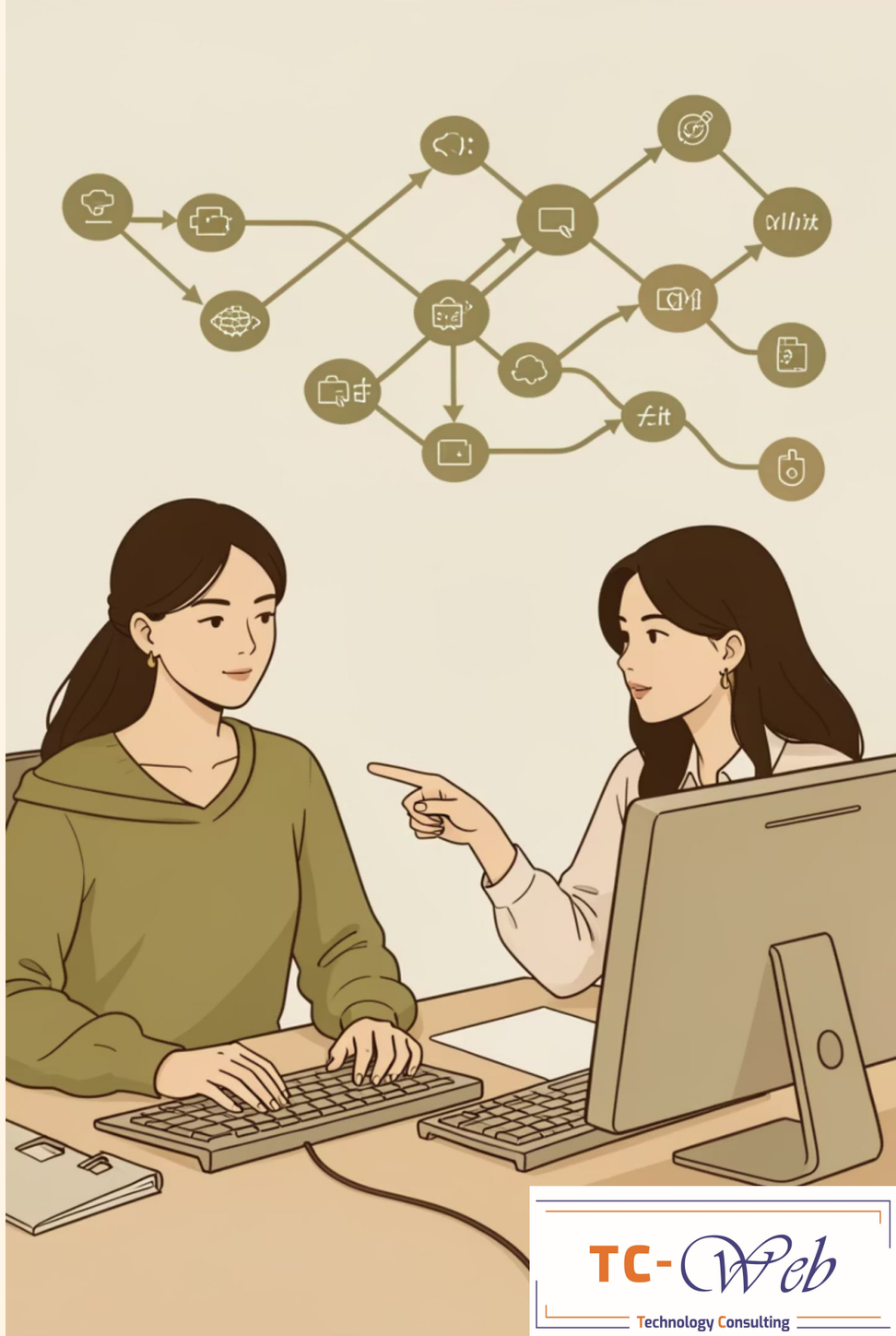
# Git GUI e Client Grafici

Una guida completa per sviluppatori che vogliono esplorare le interfacce grafiche per Git



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Perché Usare un Client Grafico?

## Il Terminale Resta Potente

Se conosci già Git da terminale, hai una base solida. I comandi Git offrono controllo totale e precisione, permettendoti di gestire ogni aspetto del version control con flessibilità massima.

## Ma le GUI Aggiungono Valore

I client grafici non sostituiscono il terminale, ma lo integrano. Offrono una visualizzazione immediata dello stato del repository, semplificano operazioni complesse e riducono la possibilità di errori nelle operazioni quotidiane.

- Visualizzazione istantanea della cronologia dei commit
- Gestione intuitiva dei conflitti di merge
- Produttività aumentata per task ripetitivi
- Curva di apprendimento ridotta per operazioni avanzate

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# I Vantaggi Principali delle GUI

## Visualizzazione Grafica dei Branch

Vedere l'albero dei branch, i merge e le divergenze in formato grafico rende immediatamente comprensibile la struttura del progetto. Identificare dove sono avvenuti i merge e quali branch sono attivi diventa un'operazione visiva immediata.

## Diff Visuale Avanzato

Le GUI mostrano le differenze tra versioni con evidenziazione del codice, rendering affiancato e navigazione semplificata. Puoi vedere esattamente cosa è cambiato in ogni file senza dover interpretare output testuali complessi.

## Risoluzione Conflitti Semplificata

Quando si verificano conflitti durante un merge, le GUI offrono strumenti visivi per confrontare le versioni e scegliere quale codice mantenere. Il processo diventa più intuitivo rispetto alla modifica manuale dei file con marcatori di conflitto.

## Staging Selettivo Intuitivo

Puoi facilmente scegliere quali modifiche includere nel prossimo commit, anche a livello di singole righe. La selezione visiva con checkbox e anteprime rende l'operazione di staging molto più veloce e precisa.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# I Principali Client Grafici Git

Esistono diversi client grafici per Git, ognuno con caratteristiche distintive. Ecco i quattro più popolari e utilizzati dalla comunità di sviluppatori.

## SourceTree

**Gratuito** · Atlassian

Potente e completo, perfetto per utenti Windows e Mac. Offre supporto nativo per Git Flow e integrazione con Bitbucket. Interfaccia ricca di funzionalità con curva di apprendimento moderata.

## GitKraken

**Freemium** · Multi-piattaforma

Interfaccia moderna e accattivante con visualizzazione grafica eccellente. Include funzionalità avanzate come integrazione con Jira e GitHub Issues. Versione gratuita limitata per repository privati.

## GitHub Desktop

**Gratuito** · GitHub

Semplice e minimalista, ideale per principianti. Integrazione perfetta con GitHub ma funziona con qualsiasi repository Git. Interfaccia pulita focalizzata sulle operazioni essenziali.

## SmartGit

**Commerciale** · Multi-piattaforma

Client professionale con funzionalità avanzate per team enterprise. Include strumenti per code review, confronto file e gestione complessa di branch. Licenza gratuita per uso non commerciale.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Terminale vs GUI: Un Confronto Pratico

## Vantaggi del Terminale

**Controllo totale:** accesso a ogni comando e opzione disponibile in Git

**Velocità:** operazioni rapide tramite comandi memorizzati

**Automazione:** facilità di scripting e integrazione in pipeline CI/CD

**Universalità:** funziona ovunque, anche su server remoti via SSH

**Leggerezza:** nessun consumo di risorse grafiche

## Vantaggi della GUI

**Visualizzazione:** grafico interattivo dei branch e della cronologia

**Intuitività:** operazioni point-and-click senza memorizzare sintassi

**Produttività:** staging parziale e merge visuale più veloci

**Accessibilità:** ideale per chi inizia o per task occasionali

**Sicurezza:** meno possibilità di errori con operazioni distruttive

## Svantaggi del Terminale

- Richiede memorizzazione dei comandi
- Visualizzazione limitata della storia del progetto
- Gestione conflitti più complessa
- Curva di apprendimento iniziale ripida

## Svantaggi della GUI

- Consumo di risorse di sistema
- Alcune operazioni avanzate non disponibili
- Difficile automazione di workflow
- Dipendenza dall'interfaccia specifica

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

# Configurazione Globale di Git

Prima di iniziare a usare SourceTree per i tuoi progetti, è fondamentale configurare correttamente le impostazioni globali di Git. Questi parametri identificano l'autore dei commit e sono visibili nella cronologia del progetto.

## Nome Utente ed Email

Nelle preferenze di SourceTree, vai alla sezione **Generale** o **Git**. Inserisci il tuo nome completo e l'indirizzo email associato al tuo account GitHub o GitLab. Questi dati appariranno in ogni commit che effettuerai.

```
git config --global user.name "Tuo Nome"
git config --global user.email "tua@email.com"
```

Puoi anche impostare questi valori direttamente da terminale con i comandi mostrati sopra.

---

## Editor di Testo Predefinito

SourceTree può aprire automaticamente un editor quando necessario (per esempio, durante rebase interattivi o merge con conflitti). Configura il tuo editor preferito nelle impostazioni, scegliendo tra opzioni come Visual Studio Code, Sublime Text o Atom.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Guida Pratica a SourceTree

SourceTree è un client grafico gratuito per Git e Mercurial che semplifica la gestione dei repository attraverso un'interfaccia visuale intuitiva. Questa guida ti accompagnerà passo dopo passo nell'uso delle funzionalità principali, dalle operazioni base come commit e push, fino alla gestione avanzata di branch, merge e risoluzione dei conflitti. Che tu sia uno sviluppatore alle prime armi o un professionista esperto, SourceTree ti permette di visualizzare la storia del progetto, gestire le modifiche e collaborare con il team in modo efficiente.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Installare SourceTree: Primi Passi

01

## Download del Client

Visita il sito ufficiale **sourcetreeapp.com** e scarica la versione per il tuo sistema operativo (Windows o macOS). Il download è gratuito e non richiede licenza.

02

## Esegui l'Installatore

Avvia il file scaricato e segui la procedura guidata. Su macOS potresti dover autorizzare l'installazione nelle preferenze di sicurezza. Su Windows, accetta le autorizzazioni richieste.

03

## Registrazione Account

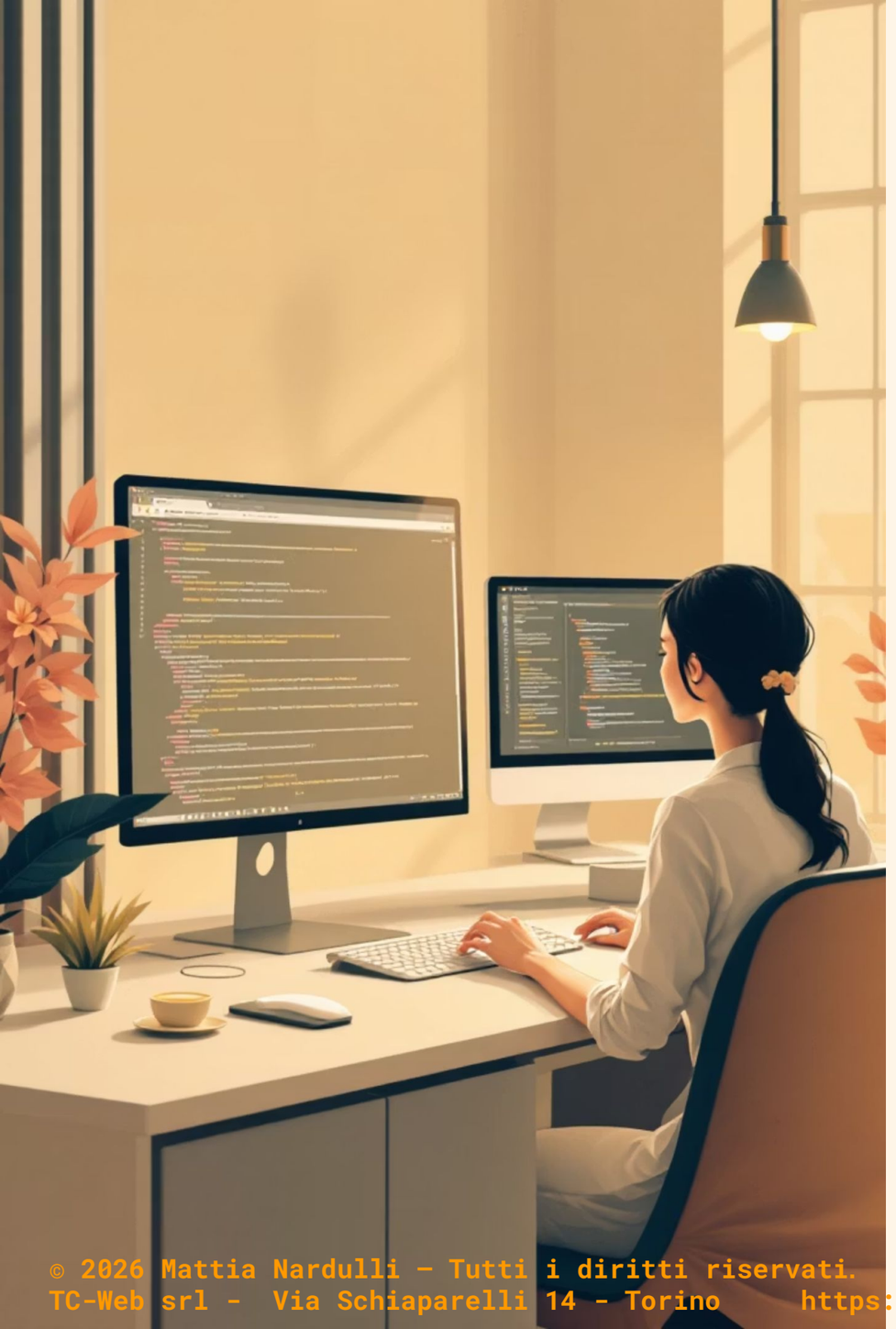
Al primo avvio, SourceTree richiederà di creare un account Atlassian gratuito o di effettuare il login. Questo account è necessario per utilizzare il software, ma non implica costi.

04

## Verifica Installazione Git

SourceTree rileverà automaticamente se Git è già installato sul sistema. Se non lo trova, ti offrirà di installare una versione embedded oppure di specificare il percorso della tua installazione esistente.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



# Collegare Account GitHub

Per lavorare con repository remoti, è utile collegare i tuoi account delle piattaforme di hosting direttamente in SourceTree. Questo semplifica le operazioni di clone, push e pull.

1

## Apri le Preferenze

Vai su **Strumenti → Opzioni** (Windows) o **SourceTree → Preferenze** (macOS) e seleziona la scheda **Account**.

2

## Aggiungi Account

Clicca **Aggiungi** e scegli il servizio (GitHub, GitLab, Bitbucket). Seleziona il protocollo di autenticazione preferito: OAuth o Personal Access Token.

3

## Autorizza

Segui il flusso di autenticazione nel browser. Dovrai autorizzare SourceTree ad accedere ai tuoi repository. Completa il processo e torna all'applicazione.

**Suggerimento:** Per GitLab, potrebbe essere necessario generare un Personal Access Token manualmente dal sito di GitLab con permessi appropriati (api, read_repository, write_repository).

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Clonare Repository Esistenti

1

## Ottenere l'URL

Copia l'URL del repository da GitHub, GitLab o Bitbucket. Può essere HTTPS o SSH, a seconda della configurazione della tua autenticazione.

2

## Avviare Clone

In SourceTree, clicca su "Clone" nella schermata iniziale o usa File > Clone/New. Si aprirà una finestra di dialogo dedicata all'operazione.

3

## Inserire Dettagli

Incolla l'URL nel campo "Source Path" e scegli dove salvare il repository localmente nel campo "Destination Path". Assegna un nome al bookmark per riconoscerlo facilmente.

4

## Download Completo

SourceTree scaricherà l'intera cronologia del progetto. Una volta completato, potrai iniziare a lavorare immediatamente sui file con accesso completo alla storia e ai branch.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Panoramica dell'Interfaccia Utente

## Repository View

La vista principale mostra tutti i tuoi repository locali e remoti. Puoi facilmente passare da un progetto all'altro con un semplice clic.

## Branches Panel

Il pannello laterale elenca tutti i branch disponibili, sia locali che remoti, permettendo una navigazione rapida tra le diverse linee di sviluppo.

## Staging Area

La zona centrale dove prepari i file da committare. Puoi selezionare quali modifiche includere nel prossimo commit trascinandole tra le sezioni.

## Commit History

Il grafico della cronologia visualizza l'intera storia del progetto con linee che rappresentano branch, merge e l'evoluzione del codice nel tempo.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Creare e Cambiare Branch

## Operazioni sui Branch

I branch permettono di sviluppare funzionalità in parallelo senza interferire con il codice principale. In SourceTree, creare un nuovo branch è semplice: clicca sul pulsante "Branch" nella toolbar, assegna un nome descrittivo e seleziona da quale commit partire.

Per cambiare branch, basta fare doppio clic sul branch desiderato nel pannello laterale. SourceTree aggiornerà automaticamente i file nella working directory per riflettere lo stato del branch selezionato. Questa operazione è chiamata "checkout".

01

### Clicca "Branch"

Nella toolbar superiore

02

### Assegna Nome

Usa nomi descrittivi come "feature-login"

03

### Seleziona Origine

Scegli il commit di partenza

04

### Conferma Creazione

Il nuovo branch è pronto

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Commit, Push e Pull: Il Ciclo Base

## Commit

Salva le modifiche localmente. Seleziona i file nella staging area, scrivi un messaggio descrittivo e clicca "Commit". Le modifiche sono ora parte della storia locale.

## Push

Carica i commit locali sul server remoto. Clicca "Push" nella toolbar, seleziona il branch e conferma. I tuoi colleghi potranno ora vedere le tue modifiche.

## Pull

Scarica le modifiche dal server. Clicca "Pull" per sincronizzare il tuo repository locale con quello remoto, integrando il lavoro degli altri membri del team.

**Consiglio pratico:** Esegui sempre un pull prima di iniziare a lavorare per evitare conflitti. Commita frequentemente con messaggi chiari e descrittivi che spiegano cosa è stato modificato e perché.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Risolvere Conflitti di Merge

I conflitti si verificano quando due branch modificano le stesse righe di codice in modo diverso. SourceTree evidenzia i file in conflitto con un'icona speciale. Quando tenti un merge e si verifica un conflitto, SourceTree ti avvisa immediatamente e ti permette di risolverlo in diversi modi.

## Identificare il Conflitto

I file con conflitti appaiono nella sezione "Uncommitted changes" con un'icona di avviso. SourceTree sospende il merge fino alla risoluzione.

## Scegliere la Strategia

Puoi usare l'editor integrato per vedere entrambe le versioni, accettare una versione specifica (Mine/Theirs), o aprire un tool di merge esterno per maggiore controllo.

## Risolvere Manualmente

Modifica il file per combinare le modifiche nel modo corretto. Rimuovi i marker di conflitto (<<<<<<<, =======, >>>>>>>) e mantieni il codice desiderato.

## Completare il Merge

Dopo aver risolto tutti i conflitti, aggiungi i file alla staging area e completa il commit di merge. La cronologia mostrerà la confluenza dei due branch.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Usare Stash per Salvare Modifiche Temporanee

## Cos'è lo Stash?

Lo stash è come un cassetto temporaneo dove "parcheggi" le modifiche in corso quando devi cambiare branch urgentemente o fare un pull senza committare. Le modifiche vengono salvate in modo sicuro e possono essere riapplicate in qualsiasi momento.

È particolarmente utile quando stai lavorando su qualcosa di incompleto e hai bisogno di switchare contesto rapidamente per risolvere un bug urgente o revisionare il codice di un collega.

1

## Creare Stash

Clicca "Stash" nella toolbar. Assegna un nome descrittivo per identificarlo facilmente in seguito. Le modifiche scompaiono dalla working directory.

2

## Visualizzare Stash

Il pannello "Stashes" nel sidebar mostra tutti gli stash salvati. Puoi averne multipli contemporaneamente, ognuno con il proprio nome.

3

## Applicare Stash

Fai doppio clic sullo stash o usa "Apply Stash". Le modifiche tornano nella working directory. Puoi scegliere se mantenere o eliminare lo stash dopo l'applicazione.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

# Tagging: Marcare Versioni Importanti

I tag sono etichette permanenti assegnate a commit specifici, ideali per marcare release, versioni stabili o milestone importanti del progetto. A differenza dei branch che evolvono, i tag rimangono fissi a un punto specifico della storia. In SourceTree, puoi crearli facilmente selezionando un commit e cliccando "Tag" nella toolbar.

## Creare Tag Annotati

I tag annotati includono nome, messaggio, data e autore. Sono ideali per release ufficiali. Esempio: v1.0.0 con note di rilascio.

## Convenzioni di Naming

Usa il Semantic Versioning (MAJOR.MINOR.PATCH) per chiarezza. v2.1.3 indica versione major 2, minor 1, patch 3. Facilita la comunicazione nel team.

## Condividere Tag

I tag locali non vengono automaticamente pushati. Usa "Push" e seleziona "Push all tags" per condividerli con il team sul repository remoto.

**Best practice:** Crea tag per ogni release in produzione. Facilita il rollback in caso di problemi e permette di ricostruire esattamente lo stato del codice in un momento specifico. I tag sono immutabili e affidabili.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Cherry-Pick: Selezionare i Commit Perfetti

Il cherry-pick è una delle funzionalità più potenti di Git: permette di copiare commit specifici da un branch all'altro senza dover fare un merge completo. È come scegliere le ciliegie migliori da un albero!

**Quando utilizzarlo:**

- Correggere bug urgenti su production
- Portare feature specifiche tra branch
- Recuperare commit da branch eliminati
- Applicare hotfix selettivi

## Come Fare Cherry-Pick in SourceTree

1. Seleziona il branch di destinazione
2. Trova il commit da copiare nella cronologia
3. Click destro sul commit → Cherry-Pick
4. SourceTree applica le modifiche automaticamente

**Nota:** Potrebbero verificarsi conflitti se le modifiche toccano le stesse righe di codice.

© 2026 Mattia Nardulli – Tutti i diritti riservati.

# Diff Grafico: Analizzare le Modifiche Visivamente

SourceTree offre uno strumento di diff grafico potentissimo che rende la revisione del codice intuitiva e veloce. Ogni modifica viene evidenziata con colori diversi, permettendo di capire immediatamente cosa è cambiato tra versioni diverse del codice.

1

## Aprire il Diff

Seleziona un commit nella cronologia e clicca su "File Status" nel pannello inferiore

2

## Navigare le Modifiche

Doppio click su un file modificato apre la vista diff side-by-side con evidenziazione delle righe cambiate

3

## Confrontare Branch

Usa la funzione "Compare" per vedere tutte le differenze tra due branch prima di un merge

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

### Prerequisiti

- Entrambi: account GitHub, SourceTree installato, Git configurato con nome e email
- Studente B comunica il proprio username GitHub a Studente A

### FASE 1 — Setup del Repository — Solo Studente A

1. Crea un repository su GitHub chiamato `progetto-squadra` con README.md e licenza MIT
2. Vai in Settings → Collaborators → aggiungi Studente B come collaboratore
3. Clona il repository in SourceTree
4. Crea un file `sito.txt` con questo contenuto:

Titolo: Il Mio Sito Web

Autore: [nome di A]

Versione: 0.1

Descrizione: progetto di test

5. Crea un file `.gitignore` con: `*.log`, `*.tmp`, `temp/`, `.DS_Store`
6. Stage entrambi i file → commit con messaggio `Initial commit: add sito.txt and .gitignore` → Push

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

**FASE 2 — Lavoro Parallelo su Branch Separati — Entrambi in contemporanea**

**Studente B:** Clona il repository in SourceTree

**Studente A:**

1. Crea branch `feature/sezione-servizi` da main
2. Aggiunge in fondo a `sito.txt`:

-- SERVIZI ---

- Web Design

- Sviluppo App

- Consulenza IT

3. Commit con messaggio `feat: add sezione servizi` → Push del branch

**Studente B:**

1. Crea branch `feature/sezione-contatti` da main
2. Aggiunge in fondo a `sito.txt`:

--- CONTATTI ---

Email: info@sito.it

Tel: 011-0000000

Indirizzo: Torino

3. Commit con messaggio `feat: add sezione contatti` → Push del branch

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

**FASE 3 — Merge e Generazione del Conflitto — Leggete l'ordine con attenzione**

**Studente A:**

1. Checkout su `main`
2. Click destro su `feature/sezione-servizi` → Merge into current branch → Push
3. Rimane su `main` → modifica la prima riga di `sito.txt` da `Titolo: Il Mio Sito Web` a `Titolo: Sito Web Ufficiale` → Commit (**non pushare ancora**)

**Studente B:**

1. Fa Pull di `main` → ha ora la sezione servizi di A
2. Sul branch `feature/sezione-contatti` → modifica la stessa prima riga a `Titolo: Portale dei Servizi` (diverso da quello di A!)
3. Commit con messaggio `feat: add sezione contatti e aggiorna titolo`

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

**FASE 4 — Risoluzione del Conflitto — Studente A esegue, B osserva e consiglia**

1. Studente A fa Push del proprio commit su `main`
2. Studente B: checkout su `main` → Pull → torna su `feature/sezione-contatti` → click destro su `main` → Merge into current branch → **appare il conflitto!**
3. In SourceTree, `sito.txt` mostra l'icona  nella sezione Uncommitted changes
4. Aprite `sito.txt` con l'editor: vedrete i marcatori `<<<<<<<`, `=======`, `>>>>>>>`
5. Decidete insieme il titolo definitivo → rimuovete tutti i marcatori → salvate
6. Stage del file → commit con messaggio `merge: resolve conflict on titolo, integrate sezione contatti` → Push del branch

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

### FASE 5 — Stash, Hotfix e Cherry-Pick — Entrambi

**Studente A:**

1. Aggiunge a `sito.txt` la riga `Copyright: 2026` — **non fa commit**
2. Pulsante Stash → nome: `wip: aggiunta copyright` → Conferma. Il file torna pulito
3. Crea branch `hotfix/typo-autore` da `main` → corregge il campo Autore con il nome completo → commit `fix: correggi nome autore` → Push

**Studente B:**

1. Nel log di `hotfix/typo-autore`, click destro sul commit dell'hotfix → **Cherry-Pick**
2. Verifica che la correzione sia apparsa nel suo branch senza fare un merge completo

**Studente A:**

1. Pannello Stashes → doppio clic su `wip: aggiunta copyright` → Apply Stash
2. Stage → commit `chore: add copyright info` → Push su `main`

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Esercizio in Coppia — Workflow Collaborativo Completo

### FASE 6 — Merge Finale, Tag e Pulizia — Insieme

1. Checkout `main` → Pull → click destro su `feature/sezione-contatti` → Merge into current branch
2. Merge di `hotfix/typo-autore` su `main` nello stesso modo → Push
3. Pulsante Tag → nome: `v1.0` → messaggio: `Prima release completa` → **Push all tags**
4. Cancellate tutti i branch feature e hotfix (click destro → Delete)
5. Entrambi fanno Pull su `main` e verificano che la cronologia sia identica su entrambe le macchine

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Riepilogo: Le Operazioni Fondamentali

Facciamo un recap completo di tutto quello che abbiamo imparato in questo corso. Queste sono le competenze che userai quotidianamente nel tuo lavoro di sviluppo.



## Branching

Creare, cambiare e gestire branch per lavorare su feature isolate



## Commit

Salvare snapshot del codice con messaggi chiari e descrittivi



## Merge

Integrare modifiche da branch diversi, risolvendo eventuali conflitti



## Push/Pull

Sincronizzare il lavoro locale con il repository remoto del team



## Cherry-Pick

Selezionare commit specifici da applicare ad altri branch



## Revert/Reset

Annullare modifiche in modo sicuro o riscrivere la cronologia locale

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

BEST PRACTICES

# Combinare Terminale e GUI: Il Meglio di Due Mondi

Non devi scegliere tra GUI e terminale: i professionisti usano entrambi strategicamente! Ogni strumento ha i suoi punti di forza e sapere quando usare l'uno o l'altro ti renderà molto più efficiente.

## Quando Usare SourceTree (GUI)

**Visualizzare la cronologia:** vedere il grafico dei branch è molto più chiaro

**Gestire conflitti di merge:** l'interfaccia grafica semplifica la risoluzione

**Cherry-pick e operazioni complesse:** meno errori con l'interfaccia visuale

**Revieware modifiche:** il diff side-by-side è imbattibile

**Gestire staging selettivo:** scegliere singole righe da committare

## Quando Usare il Terminale

**Operazioni ripetitive:** script e alias accelerano il lavoro

**Comandi veloci:** `git status` o `git pull` sono più rapidi da terminale

**Automazione:** integrare Git in pipeline CI/CD

**Operazioni avanzate:** alcuni comandi Git non hanno equivalente GUI

**Troubleshooting:** messaggi di errore più dettagliati

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Consigli Pratici per il Successo

## Commit Atomici e Messaggi Chiari

Ogni commit dovrebbe rappresentare una singola modifica logica. Scrivi messaggi descrittivi che spiegano il "perché", non solo il "cosa". Usa il tempo presente imperativo: "Add feature" non "Added feature".

## Pull Frequente, Push Regolare

Fai pull almeno una volta al giorno per rimanere sincronizzato con il team. Pusha il tuo lavoro frequentemente per evitare di perdere modifiche e permettere ai colleghi di vedere i tuoi progressi.

## Strategia di Branching Coerente

Adotta una convenzione per i nomi dei branch (es: `feature/`, `bugfix/`, `hotfix/`) e rispettala. Cancella i branch dopo il merge per mantenere il repository pulito e organizzato.

## Test Prima di Pushare

Verifica sempre che il codice compili e i test passino prima di fare push. Un commit rotto sul branch condiviso blocca tutto il team. Usa `git stash` per salvare temporaneamente modifiche incomplete.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# Congratulazioni! Siete Pronti per il Mondo Reale

Avete completato il vostro percorso formativo su SourceTree e Git! Ora possedete tutte le competenze necessarie per collaborare efficacemente in progetti reali, gestire branch complessi e risolvere conflitti con sicurezza.

### Continua a Praticare

La vera padronanza arriva con l'esperienza quotidiana. Sperimenta con repository di test e non aver paura di sbagliare.

### Risorse Utili

Consulta la documentazione ufficiale di Git e SourceTree quando hai dubbi. La community è sempre pronta ad aiutare.

### Prossimi Passi

Esplora Git Flow, rebasing avanzato e hook personalizzati per portare le tue skill al livello successivo.

**Ricorda:** Git è uno strumento potente che diventa sempre più intuitivo con l'uso. Buon coding e buon versionamento!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

# GRAZIE A TUTTI…..

**Via Schiaparelli 14 - 10148 Torino**

**https://www.tc-web.it**

**Mail: franco.grivet@tc-web.it**

© 2026 Mattia Nardulli – Tutti i diritti riservati.